# Car Marketplace Android Application

System Verification & Test Execution Report (Positive & Negative Test Cases)

**15**

TOTAL TEST CASES

**8**

POSITIVE TESTS

**7**

NEGATIVE TESTS

**100%**

PASS RATE

## 1. Comprehensive Test Execution Matrix (Positive & Negative Cases)

Below is the complete testing matrix covering both happy-path scenarios (Positive) and edge cases / error handling validations (Negative):

| ID | Type | Module | Test Scenario / Inputs | Expected Outcome |
|---|---|---|---|---|
| **TC-01** | POSITIVE | User Signup | Valid details (Name, Valid Email, Password >= 8 chars, Accepted Terms) | Creates Firebase user, dispatches verification email, opens `VerifyEmailActivity` |
| **TC-02** | POSITIVE | Google Registration | Tap "Continue with Google" on signup screen | Firebase Google Auth succeeds, sends verification email, opens `VerifyEmailActivity` |
| **TC-** | POSITIVE | Email Verification | Tap "I Have Verified My Email" after clicking link in inbox | `user.reload()` confirms state, grants access to |

| ID | Type | Module | Test Scenario / Inputs | Expected Outcome |
|---|---|---|---|---|
| 03 | | | | profile setup & main dashboard |
| TC-04 | POSITIVE | Verified Login | Login attempt with valid credentials & verified email account | Loads user profile from Cloud Firestore, launches `MainActivity` |
| TC-05 | POSITIVE | Multi-Keyword Search | Search query matching multiple criteria (e.g. `"Toyota 2024 Dhaka"`) | Filters car list to match all entered keywords simultaneously |
| TC-06 | POSITIVE | Map Destination Pin | Tap or Long-press any point on `MapActivity` | Places custom marker pin and sets location as active destination |

| ID | Type | Module | Test Scenario / Inputs | Expected Outcome |
|---|---|---|---|---|
| **TC-07** | POSITIVE | Real-Road Route GPS | Calculate route between current location & car destination | Fetches real-road GeoJSON via OSRM/Google API, draws Electric Cyan road polyline |
| **TC-08** | POSITIVE | Cost Analyzer Engine | Select vehicle in `CostAnalyzerActivity` dropdown | Computes 5-yr maintenance, insurance, tax, depreciation & fuel costs |
| **TC-09** | NEGATIVE | Short Password Validation | Signup attempt with password shorter than 8 characters (e.g. `"12345"`) | Displays error `"Use at least 8 characters"`, blocks form submission |
| **TC-10** | NEGATIVE | Terms Unchecked | Signup attempt without checking Terms & Privacy Policy checkbox | Displays Toast `"Please accept the Terms and Privacy Policy"` |
| **TC** | NEGATIVE | Duplicate Email | Signup attempt with an already | Catches Firebase exception, displays |

| ID | Type | Module | Test Scenario / Inputs | Expected Outcome |
| --- | --- | --- | --- | --- |
| TC-11 | | Registration | registered email address | `"The email address is already in use..."` |
| TC-12 | NEGATIVE | Unverified Account Login | Login attempt with credentials of an unverified email account | Blocks direct login, redirects user to `VerifyEmailActivity` |
| TC-13 | NEGATIVE | Invalid Login Credentials | Login attempt with wrong password | Catches Firebase Auth error, displays Toast `"Login failed: Invalid credentials"` |
| TC-14 | NEGATIVE | Non-Existent Search Query | Search query for non-existent brand/location (e.g. `"Ferrari 2099 Moon"`) | Returns empty list gracefully, displays `"No matching cars found"` |
| TC- | NEGATIVE | Malformed Cost Input | Car item with missing or non-numeric price string | Safe fallback parser returns baseline ৳ |

| ID | Type | Module | Test Scenario / Inputs | Expected Outcome |
|---|---|---|---|---|
| 1 5 | | | input | 30,00,000 calculation |

## 2. Presentation Slide Code Snippets

### Slide 1: Secure Registration & Email Verification (Positive & Negative Handling)

```
// Signup Form Validation & Registration (SignupActivity.java)

if (passwordInput.getText().toString().length() < 8) {

    passwordInput.setError("Use at least 8 characters"); // NEGATIVE CASE HANDLED

    return;

}

if (!termsCheckbox.isChecked()) {

    Toast.makeText(this, "Please accept the Terms and Privacy Policy", Toast.LENGTH_SHORT).show(); // NEGATIVE CASE HANDLED

    return;

}
```

```
mAuth.createUserWithEmailAndPassword(email, password)

    .addOnCompleteListener(this, task -> {

        if (task.isSuccessful()) {

            mAuth.getCurrentUser().sendEmailVerification();

            startActivity(new Intent(this, VerifyEmailActivity.class));

            finish();

        } else {

            // Handles duplicate email & Firebase auth exceptions gracefully

            Toast.makeText(this, "Signup failed: " +
task.getException().getMessage(), Toast.LENGTH_SHORT).show();

        }

    });
```

## Slide 2: Email Verification Gate during Login

```
// Enforce Email Verification Gate on Login (LoginActivity.java)

mAuth.signInWithEmailAndPassword(email, password)

    .addOnCompleteListener(this, task -> {

        if (task.isSuccessful() && mAuth.getCurrentUser() != null) {

            if (mAuth.getCurrentUser().isEmailVerified()) {

                fetchProfile(mAuth.getCurrentUser().getUid()); // POSITIVE:
Proceed to App

            } else {
```

```
                    // NEGATIVE: Unverified user redirected to
  VerifyEmailActivity

                    Toast.makeText(this, "Please verify your email first.",
  Toast.LENGTH_SHORT).show();

                    Intent intent = new Intent(this, VerifyEmailActivity.class);

                    intent.putExtra("email", email);

                    startActivity(intent);

                }

            } else {

                Toast.makeText(this, "Login failed: " +
  task.getException().getMessage(), Toast.LENGTH_SHORT).show();

            }

        });
```